ОТЧЁТ

по лабораторной работе №11

«Реляционные и нереляционные базы данных, кэш и CDN»

Вариант 20. Интернет-провайдер, сущность Tariff

1. Цель работы

Научиться проектировать реляционную схему, писать параметризованный SQL, проводить транзакцию, сохранять документы расчётов, кэшировать повторные чтения и отделять статику тарифа от оперативных данных.

2. Постановка индивидуального задания

Вариант 20. Основная сущность — тариф. В документном хранилище лежат результаты расчётов, через CDN предполагается отдавать изображения тарифов. Реляционная модель содержит не меньше трёх связанных таблиц.

SQL написан вручную для SQLite. Кэш и документы в локальном запуске используют совместимые in-memory реализации Redis и MongoDB с тем же программным интерфейсом, потому что отдельные серверы в окружении не поднимались.

3. Краткие теоретические сведения

В реляционной базе данные разложены по таблицам. Первичный ключ однозначно указывает строку, внешний ключ связывает её с другой таблицей. Связь «многие ко многим» между тарифом и услугой идёт через таблицу tariff_services. Нормализация убирает повторение региона и услуги в каждой строке тарифа.

Параметризованный запрос передаёт значения отдельно от текста SQL, через знак ?. Склейка строки с пользовательским вводом открывает SQL-инъекцию. Транзакция либо фиксирует все шаги, либо откатывает их. Индекс ускоряет отбор, например по скорости.

Документная база хранит расчёт целиком, без заранее жёсткой таблицы. Cache-aside сначала смотрит кэш, при промахе читает источник и кладёт значение с TTL. Картинку тарифа логично отдавать с CDN: она одинакова для всех и редко меняется, а версия в имени файла позволяет не сбрасывать кэш всего сайта.

4. Описание программы

Таблицы: regions, tariffs, services и tariff_services. При первом запуске загружаются регионы, услуги и не меньше десяти тарифов. Запросы демонстрируют SELECT, WHERE, ORDER BY, JOIN, LEFT JOIN, GROUP BY, HAVING, COUNT, AVG, подзапрос и несколько условий. EXPLAIN QUERY PLAN показывает использование индекса по скорости.

Добавление тарифа вместе со связью на услугу идёт одной транзакцией. Стоимость «плата плюс услуги» сначала считается SQL, повторно берётся из кэша. Документ расчёта сохраняется в коллекции calculations. В каталоге static лежат SVG тарифа и описание политики Cache-Control.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — main.py

```
"""
Лабораторная работа №11. Вариант 20.
SQLite, документные расчёты, кэш и статика тарифов.
"""

import sys

from cache_and_docs import CalculationStore, PriceCache, cdn_headers, write_static
from database import add_tariff_with_service, connect, explain_index, init_db, showcase_queries

if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")



def read_line(prompt):
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
    print(f"{prompt}{text}")
    return text.strip()



def print_menu():
    print()
    print("===== Хранение тарифов =====")
    print("1. Показать SQL-запросы")
    print("2. План запроса по индексу скорости")
    print("3. Добавить тариф с услугой (транзакция)")
    print("4. Расчёт стоимости через кэш")
    print("5. Сохранить расчёт в документную БД")
    print("6. Статика и CDN")
    print("0. Выход")
    print()



def price_from_sql(conn, name):
    row = conn.execute("SELECT price FROM tariffs WHERE name = ?", (name,)).fetchone()
    if row is None:
        raise ValueError("тариф не найден")
    services = conn.execute(
```

```
        """
        SELECT COALESCE(SUM(s.price), 0)
        FROM tariff_services ts
        JOIN services s ON s.id = ts.service_id
        JOIN tariffs t ON t.id = ts.tariff_id
        WHERE t.name = ?
        """,
        (name,),
    ).fetchone()[0]
    return row["price"] + services


def main():
    conn = connect()
    init_db(conn)
    cache = PriceCache()
    docs = CalculationStore()
    print("База открыта, внешние ключи включены, тестовые тарифы загружены.")

    while True:
        print_menu()
        choice = read_line("Выберите действие: ")
        try:
            if choice == "1":
                print("\nЗапросы выполняются с параметрами ?, не через склейку строк.")
                for title, count, preview in showcase_queries(conn):
                    print(f"\n{title}: строк {count}")
                    for row in preview:
                        print(" ", row)
            elif choice == "2":
                print("EXPLAIN QUERY PLAN для speed >= 100:")
                for row in explain_index(conn):
                    print(" ", row)
                print("Индекс idx_tariffs_speed позволяет не просматривать всю таблицу при отборе по
скорости.")
            elif choice == "3":
                name = read_line("Название: ")
                speed = int(read_line("Скорость: "))
                price = float(read_line("Цена: ").replace(",", "."))
                region = read_line("Регион (Москва/Казань/Тула): ")
                service = read_line("Услуга (белый IP/антивирус/ТВ/статический IP): ")
                item_id = add_tariff_with_service(conn, name, speed, price, region, service)
                print(f"Транзакция зафиксирована, id = {item_id}")
            elif choice == "4":
                name = read_line("Название тарифа: ")
                key = f"price:{name}"
                total, source = cache.get_total(key, lambda: price_from_sql(conn, name))
                print(f"Сумма {total:.0f} руб., источник: {source}")
                total2, source2 = cache.get_total(key, lambda: price_from_sql(conn, name))
                print(f"Повторный запрос {total2:.0f} руб., источник: {source2}")
                print(f"Попадания в кэш: {cache.hits}, промахи: {cache.misses}, TTL 60 с")
            elif choice == "5":
                name = read_line("Название тарифа: ")
                total = price_from_sql(conn, name)
                docs.save(name, total, ["базовая плата", "услуги"])
                saved = docs.latest(name)
                print("Документ расчёта:", {k: saved[k] for k in ("tariff", "total", "services")})
            elif choice == "6":
                path = write_static()
                print("Файл изображения:", path)
                print("Заголовки CDN:", cdn_headers())
                print("Картинку тарифа отдают с CDN: она публичная и меняется редко.")
                print("Версия в имени файла (home100.v1.svg) позволяет не сбрасывать кэш всего сайта.")
            elif choice == "0":
                print("Работа программы завершена.")
                break
            else:
                print("Ошибка: выберите пункт от 0 до 6.")
        except (ValueError, Exception) as exc:
            if isinstance(exc, (EOFError, KeyboardInterrupt)):
                raise
            print(f"Ошибка: {exc}")
        except EOFError:
            break
    conn.close()
```

```
if __name__ == "__main__":
    main()
```

Листинг — cache_and_docs.py

```
"""Документное хранилище расчётов и кэш cache-aside."""

import json
import os

import fakeredis
import mongomock

BASE = os.path.dirname(os.path.abspath(__file__))



class CalculationStore:
    """Документы результатов расчёта. Для запуска без сервера MongoDB используется mongomock."""

    def __init__(self):
        self.client = mongomock.MongoClient()
        self.collection = self.client["provider"]["calculations"]

    def save(self, tariff_name, total, services):
        document = {"tariff": tariff_name, "total": total, "services": services}
        self.collection.insert_one(document)
        return document

    def latest(self, tariff_name):
        return self.collection.find_one({"tariff": tariff_name}, sort=[("_id", -1)])



class PriceCache:
    """Cache-aside: сначала Redis, при промахе — расчёт и запись с TTL."""

    def __init__(self):
        self.client = fakeredis.FakeRedis(decode_responses=True)
        self.hits = 0
        self.misses = 0

    def get_total(self, key, calculator):
        cached = self.client.get(key)
        if cached is not None:
            self.hits += 1
            return float(cached), "cache"
        self.misses += 1
        value = calculator()
        self.client.setex(key, 60, str(value))
        return value, "db"



def cdn_headers():
    """Заголовки, которые ставят перед статикой тарифа на CDN."""
    return {
        "Cache-Control": "public, max-age=86400",
        "Content-Type": "image/svg+xml",
    }



def write_static():
    folder = os.path.join(BASE, "static", "tariffs")
    os.makedirs(folder, exist_ok=True)
    svg = """<svg xmlns="http://www.w3.org/2000/svg" width="320" height="80">
<rect width="320" height="80" fill="#123"/>
<text x="16" y="48" fill="white" font-size="22">Тариф 100 Мбит/с</text>
</svg>
"""
    path = os.path.join(folder, "home100.svg")
    with open(path, "w", encoding="utf-8") as file:
        file.write(svg)
```

```
    versioned = os.path.join(folder, "home100.v1.svg")
    with open(versioned, "w", encoding="utf-8") as file:
        file.write(svg)
    note = os.path.join(BASE, "static", "cache-policy.json")
    with open(note, "w", encoding="utf-8") as file:
        json.dump(
            {
                "resource": "/static/tariffs/home100.v1.svg",
                "why_cdn": "картинка тарифа одинакова для всех абонентов и редко меняется",
                "cache_control": "public, max-age=86400",
                "versioning": "новое изображение публикуется как home100.v2.svg, старый URL не
инвалидируют вручную",
            },
            file,
            ensure_ascii=False,
            indent=2,
        )
    return path
```

Листинг — database.py

```
"""SQLite-схема тарифов. SQL написан вручную, запросы параметризованы."""

import os
import sqlite3

BASE = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(BASE, "data", "provider.db")



def connect():
    os.makedirs(os.path.dirname(DB_PATH), exist_ok=True)
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    return conn



def init_db(conn):
    conn.executescript(
        """
        CREATE TABLE IF NOT EXISTS regions (
            id INTEGER PRIMARY KEY,
            name TEXT NOT NULL UNIQUE
        );
        CREATE TABLE IF NOT EXISTS tariffs (
            id INTEGER PRIMARY KEY,
            name TEXT NOT NULL,
            speed INTEGER NOT NULL,
            price REAL NOT NULL,
            region_id INTEGER NOT NULL,
            status TEXT NOT NULL,
            FOREIGN KEY (region_id) REFERENCES regions(id)
        );
        CREATE TABLE IF NOT EXISTS services (
            id INTEGER PRIMARY KEY,
            name TEXT NOT NULL,
            price REAL NOT NULL
        );
        CREATE TABLE IF NOT EXISTS tariff_services (
            tariff_id INTEGER NOT NULL,
            service_id INTEGER NOT NULL,
            PRIMARY KEY (tariff_id, service_id),
            FOREIGN KEY (tariff_id) REFERENCES tariffs(id),
            FOREIGN KEY (service_id) REFERENCES services(id)
        );
        CREATE INDEX IF NOT EXISTS idx_tariffs_speed ON tariffs(speed);
        CREATE INDEX IF NOT EXISTS idx_tariffs_region ON tariffs(region_id);
        """
    )
    count = conn.execute("SELECT COUNT(*) FROM tariffs").fetchone()[0]
    if count:
        return
    regions = ["Москва", "Казань", "Тула"]
    for name in regions:
        conn.execute("INSERT INTO regions(name) VALUES (?)", (name,))
```

```
    services = [("белый IP", 200), ("статический IP", 150), ("антивирус", 100), ("ТВ", 250)]
    for name, price in services:
        conn.execute("INSERT INTO services(name, price) VALUES (?, ?)", (name, price))
    samples = [
        ("Домашний 50", 50, 350, 1, "активен"),
        ("Домашний 100", 100, 500, 1, "активен"),
        ("Домашний 300", 300, 800, 1, "активен"),
        ("Домашний 500", 500, 1100, 2, "активен"),
        ("Корпоративный 100", 100, 1500, 2, "активен"),
        ("Корпоративный 500", 500, 2500, 1, "активен"),
        ("Мобильный 20", 20, 250, 3, "активен"),
        ("Мобильный 50", 50, 400, 3, "архив"),
        ("Ночной 100", 100, 300, 3, "активен"),
        ("Безлимит 1 Гбит", 1000, 1800, 1, "активен"),
        ("Архивный 10", 10, 150, 2, "архив"),
    ]
    for row in samples:
        conn.execute(
            "INSERT INTO tariffs(name, speed, price, region_id, status) VALUES (?, ?, ?, ?, ?)",
            row,
        )
    links = [(2, 1), (2, 3), (6, 1), (6, 2), (10, 4), (4, 3)]
    for tariff_id, service_id in links:
        conn.execute(
            "INSERT INTO tariff_services(tariff_id, service_id) VALUES (?, ?)",
            (tariff_id, service_id),
        )
    conn.commit()


def showcase_queries(conn):
    """Набор запросов, который требует методичка."""
    queries = [
        ("SELECT", "SELECT id, name, speed, price FROM tariffs"),
        ("WHERE", "SELECT name, price FROM tariffs WHERE status = ?", ("активен",)),
        ("ORDER BY", "SELECT name, speed FROM tariffs ORDER BY speed DESC"),
        (
            "JOIN",
            """
            SELECT t.name, r.name AS region
            FROM tariffs t JOIN regions r ON r.id = t.region_id
            """,
        ),
        (
            "LEFT JOIN",
            """
            SELECT t.name, s.name
            FROM tariffs t
            LEFT JOIN tariff_services ts ON ts.tariff_id = t.id
            LEFT JOIN services s ON s.id = ts.service_id
            """,
        ),
        (
            "GROUP BY",
            """
            SELECT r.name, COUNT(t.id) AS cnt
            FROM regions r LEFT JOIN tariffs t ON t.region_id = r.id
            GROUP BY r.name
            """,
        ),
        (
            "HAVING",
            """
            SELECT r.name, COUNT(t.id) AS cnt
            FROM regions r JOIN tariffs t ON t.region_id = r.id
            GROUP BY r.name
            HAVING COUNT(t.id) >= 3
            """,
        ),
        ("COUNT", "SELECT COUNT(*) FROM tariffs"),
        ("AVG", "SELECT AVG(price) FROM tariffs WHERE status = ?", ("активен",)),
        ("подзапрос", "SELECT name FROM tariffs WHERE price > (SELECT AVG(price) FROM tariffs)"),
        (
            "несколько условий",
            "SELECT name, speed, price FROM tariffs WHERE speed >= ? AND status = ?",
            (100, "активен"),
        ),
```

```
    ]
    report = []
    for item in queries:
        title, sql = item[0], item[1]
        params = item[2] if len(item) > 2 else ()
        rows = conn.execute(sql, params).fetchall()
        preview = [tuple(row) for row in rows[:4]]
        report.append((title, len(rows), preview))
    return report



def add_tariff_with_service(conn, name, speed, price, region_name, service_name):
    """Транзакция: тариф и связь с услугой фиксируются вместе."""
    try:
        region = conn.execute("SELECT id FROM regions WHERE name = ?", (region_name,)).fetchone()
        service = conn.execute("SELECT id FROM services WHERE name = ?", (service_name,)).fetchone()
        if region is None or service is None:
            raise ValueError("регион или услуга не найдены")
        cursor = conn.execute(
            "INSERT INTO tariffs(name, speed, price, region_id, status) VALUES (?, ?, ?, ?, ?)",
            (name, speed, price, region["id"], "активен"),
        )
        conn.execute(
            "INSERT INTO tariff_services(tariff_id, service_id) VALUES (?, ?)",
            (cursor.lastrowid, service["id"]),
        )
        conn.commit()
        return cursor.lastrowid
    except Exception:
        conn.rollback()
        raise



def explain_index(conn):
    plan = conn.execute(
        "EXPLAIN QUERY PLAN SELECT * FROM tariffs WHERE speed >= ?",
        (100,),
    ).fetchall()
    return [tuple(row) for row in plan]
```

Листинг — tests\test_db.py

```
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

import database
from cache_and_docs import PriceCache
from database import add_tariff_with_service, init_db, showcase_queries



def fresh(tmp_path, monkeypatch):
    monkeypatch.setattr(database, "DB_PATH", str(tmp_path / "t.db"))
    conn = database.connect()
    init_db(conn)
    return conn



def test_seed_count(tmp_path, monkeypatch):
    conn = fresh(tmp_path, monkeypatch)
    assert conn.execute("SELECT COUNT(*) FROM tariffs").fetchone()[0] >= 10



def test_parameterized_filter(tmp_path, monkeypatch):
    conn = fresh(tmp_path, monkeypatch)
    rows = conn.execute(
        "SELECT name FROM tariffs WHERE status = ? AND speed >= ?",
        ("активен", 500),
    ).fetchall()
    assert rows
```

```
def test_transaction_links_service(tmp_path, monkeypatch):
    conn = fresh(tmp_path, monkeypatch)
    item_id = add_tariff_with_service(conn, "Тестовый", 100, 10, "Тула", "ТВ")
    link = conn.execute(
        "SELECT service_id FROM tariff_services WHERE tariff_id = ?",
        (item_id,),
    ).fetchone()
    assert link is not None



def test_queries_run(tmp_path, monkeypatch):
    conn = fresh(tmp_path, monkeypatch)
    report = showcase_queries(conn)
    assert len(report) >= 10



def test_cache_second_hit(tmp_path, monkeypatch):
    calls = {"n": 0}

    def calc():
        calls["n"] += 1
        return 42

    cache = PriceCache()
    cache.get_total("k", calc)
    cache.get_total("k", calc)
    assert calls["n"] == 1
    assert cache.hits == 1
```

6. Примеры работы программы

Снимки показывают набор SQL-запросов, план по индексу, кэш стоимости, документ расчёта, транзакцию и пояснение по статике.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python11> python main.py

База открыта, внешние ключи включены, тестовые тарифы загружены.

===== Хранение тарифов =====
1. Показать SQL-запросы
2. План запроса по индексу скорости
3. Добавить тариф с услугой (транзакция)
4. Расчёт стоимости через кэш
5. Сохранить расчёт в документную БД
6. Статика и CDN
0. Выход

Выберите действие: 1

Запросы выполняются с параметрами ?, не через склейку строк.

SELECT: строк 12
  (1, 'Домашний 50', 50, 350.0)
  (2, 'Домашний 100', 100, 500.0)
  (3, 'Домашний 300', 300, 800.0)
  (4, 'Домашний 500', 500, 1100.0)

WHERE: строк 10
  ('Домашний 50', 350.0)
  ('Домашний 100', 500.0)
  ('Домашний 300', 800.0)
  ('Домашний 500', 1100.0)

ORDER BY: строк 12
  ('Безлимит 1 Гбит', 1000)
  ('Корпоративный 500', 500)
  ('Домашний 500', 500)
  ('Домашний 300', 300)

JOIN: строк 12
  ('Домашний 50', 'Москва')
  ('Домашний 100', 'Москва')
  ('Домашний 300', 'Москва')
  ('Домашний 500', 'Казань')

LEFT JOIN: строк 14
  ('Домашний 50', None)
  ('Домашний 100', 'белый IP')
  ('Домашний 100', 'антивирус')
  ('Домашний 300', None)

GROUP BY: строк 3
  ('Казань', 3)
```

Рисунок 1 – Основные SQL-запросы по тарифам, регионам и услугам

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python11> python main.py

База открыта, внешние ключи включены, тестовые тарифы загружены.

===== Хранение тарифов =====
1. Показать SQL-запросы
2. План запроса по индексу скорости
3. Добавить тариф с услугой (транзакция)
4. Расчёт стоимости через кэш
5. Сохранить расчёт в документную БД
6. Статика и CDN
0. Выход

Выберите действие: 2
EXPLAIN QUERY PLAN для speed >= 100:
  (3, 0, 203, 'SEARCH tariffs USING INDEX idx_tariffs_speed (speed>?)')
Индекс idx_tariffs_speed позволяет не просматривать всю таблицу при отборе по скорости.

===== Хранение тарифов =====
1. Показать SQL-запросы
2. План запроса по индексу скорости
3. Добавить тариф с услугой (транзакция)
4. Расчёт стоимости через кэш
5. Сохранить расчёт в документную БД
6. Статика и CDN
0. Выход

Выберите действие: 4
Название тарифа: Домашний 100
Сумма 800 руб., источник: db
Повторный запрос 800 руб., источник: cache
Попадания в кэш: 1, промахи: 1, TTL 60 с

===== Хранение тарифов =====
1. Показать SQL-запросы
2. План запроса по индексу скорости
3. Добавить тариф с услугой (транзакция)
4. Расчёт стоимости через кэш
5. Сохранить расчёт в документную БД
6. Статика и CDN
0. Выход

Выберите действие: 5
Название тарифа: Домашний 100
Документ расчёта: {'tariff': 'Домашний 100', 'total': 800.0, 'services': ['базовая плата', 'услуги']}

===== Хранение тарифов =====
1. Показать SQL-запросы
2. План запроса по индексу скорости
3. Добавить тариф с услугой (транзакция)
4. Расчёт стоимости через кэш
5. Сохранить расчёт в документную БД
6. Статика и CDN
0. Выход

Выберите действие: 0
Работа программы завершена.
```

Рисунок 2 – План запроса по индексу скорости и повторное чтение из кэша

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python11> python main.py

База открыта, внешние ключи включены, тестовые тарифы загружены.

===== Хранение тарифов =====
1. Показать SQL-запросы
2. План запроса по индексу скорости
3. Добавить тариф с услугой (транзакция)
4. Расчёт стоимости через кэш
5. Сохранить расчёт в документную БД
6. Статика и CDN
0. Выход

Выберите действие: 3
Название: Ночной 200
Скорость: 200
Цена: 400
Регион (Москва/Казань/Тула): Тула
Услуга (белый IP/антивирус/ТВ/статический IP): ТВ
Транзакция зафиксирована, id = 13

===== Хранение тарифов =====
1. Показать SQL-запросы
2. План запроса по индексу скорости
3. Добавить тариф с услугой (транзакция)
4. Расчёт стоимости через кэш
5. Сохранить расчёт в документную БД
6. Статика и CDN
0. Выход

Выберите действие: 6
Файл изображения: C:\Users\Shmurdyakich\Desktop\pythonALL\python11\static\tariffs\home100.svg
Заголовки CDN: {'Cache-Control': 'public, max-age=86400', 'Content-Type': 'image/svg+xml'}
Картинку тарифа отдают с CDN: она публичная и меняется редко.
Версия в имени файла (home100.v1.svg) позволяет не сбрасывать кэш всего сайта.

===== Хранение тарифов =====
1. Показать SQL-запросы
2. План запроса по индексу скорости
3. Добавить тариф с услугой (транзакция)
4. Расчёт стоимости через кэш
5. Сохранить расчёт в документную БД
6. Статика и CDN
0. Выход

Выберите действие: 0
Работа программы завершена.
```

Рисунок 3 – Транзакция добавления тарифа и статика для CDN

7. Ответы на контрольные вопросы

1. Что такое первичный и внешний ключ?

Первичный ключ уникален в своей таблице. Внешний ссылается на первичный ключ другой таблицы и связывает, например, тариф с регионом.

2. Зачем промежуточная таблица?

У тарифа несколько услуг, и одна услуга входит в несколько тарифов. Пара идентификаторов в tariff_services хранит эту связь.

3. Почему нельзя подставлять ввод в текст SQL?

Пользователь может передать фрагмент запроса и изменить его смысл. Параметры отделяют данные от команды.

4. Что даёт транзакция?

Тариф и его связь с услугой сохраняются вместе. Если второй шаг не удался, откатывается и первый.

5. Чем документ расчёта отличается от строки тарифа?

Тариф в SQLite нормализован. Готовый расчёт удобнее хранить одним документом: название, сумма и состав.

6. Как работает cache-aside?

Сначала читается ключ кэша. Если его нет, значение считается из базы и записывается с временем жизни. Повтор приходит уже из кэша.

7. Что отдавать через CDN?

Изображение тарифа и другую редко меняющуюся статику. Оперативные цены и статусы остаются в базе приложения.

8. Вывод

В лабораторной работе №11 тарифы варианта 20 разложены по связанным таблицам SQLite, запросы параметризованы, добавление тарифа с услугой выполняется транзакцией. Результаты расчёта лежат в документной коллекции, повторная стоимость берётся из кэша, а изображение тарифа описано как статический ресурс с версией и Cache-Control.